COP 6611 – Graduate Operating Systems

Project 2: System Calls, User Utilities (sort & grep), and Process Schedulers in xv6

1. Group Members & Project Continuity

- **Group Members:** Mandar Dhamale, Nadika Poudel, Sakina Rao
- **Contact Email:** mandardhamale@usf.edu

- **Group Continuity Status:** Yes. Our group members are the exact same as Project 1.

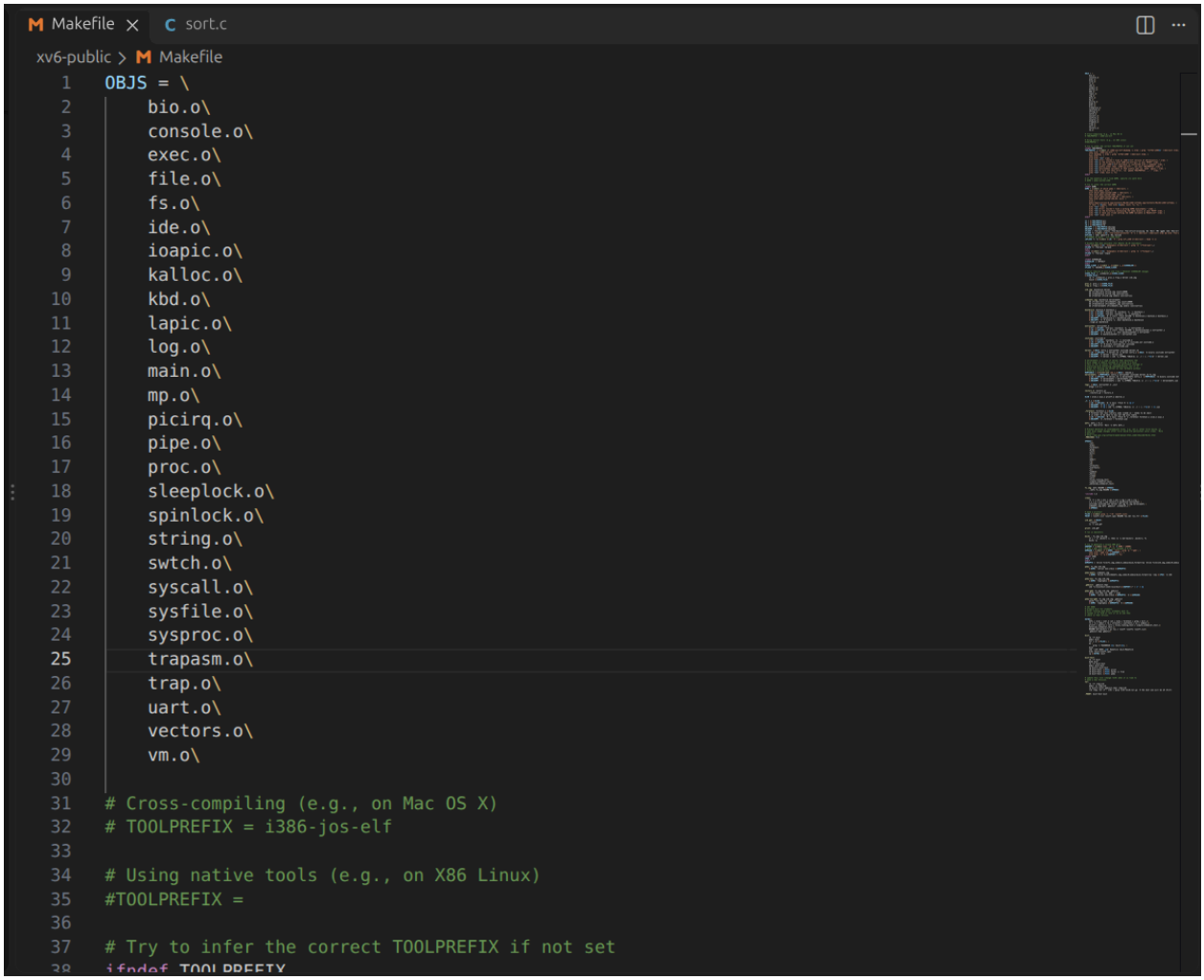
2. System & Development Environment

- **Operating System:** Ubuntu 24.04 LTS (64-bit Linux, x86_64 architecture)
- **Processor / Target Architecture:** x86_64 host processor compiling 32-bit x86 (elf_i386) target binaries for xv6
- **Compiler & Toolchain:** GNU GCC (gcc-multilib) with flags -m32 -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall
- **Emulator:** QEMU (qemu-system-i386) running with 2 virtual CPUs (-smp 2) and 512 MB RAM
- **Build Tool:** GNU Make (make)
- **Special Installation & Build Steps:** Configured 32-bit cross-compilation libraries (gcc-multilib) and updated the Makefile to support compile-time scheduler selection via SCHEDULER=DEFAULT, SCHEDULER=SJF (or SCHEDULER=[SJF]), and SCHEDULER=PRIORITYRR (or SCHEDULER=[PRIORITYRR]). Added a .scheduler_$(SCHED_CLEAN) dependency target so switching the SCHEDULER flag automatically recompiles proc.o, trap.o, kernel, and xv6.img.

3. Part 1: Adding sort and grep User Programs

3A. Implementing sort (sort.c)

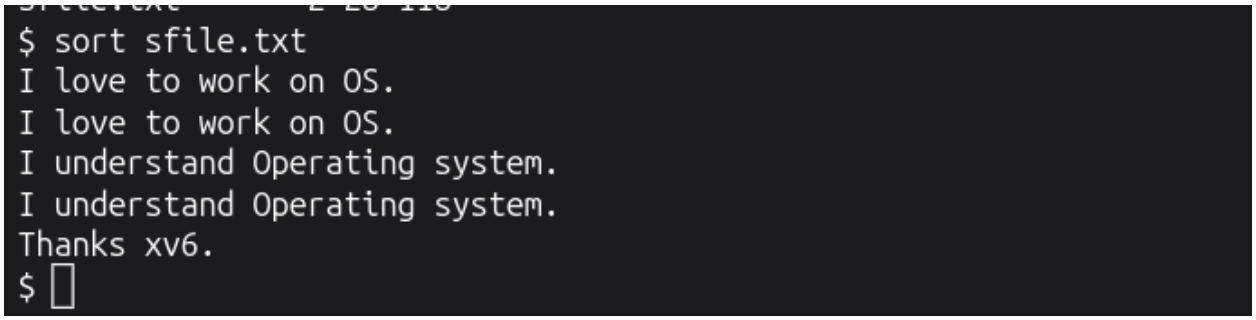
We implemented sort.c and added _sort to UPROGS and sort.c to EXTRA in the Makefile. The program reads lines from either specified file arguments or standard input (fd = 0) when piped, dynamically allocates line buffers, sorts them using insertion sort with custom comparator compare_lines(), and prints the output to stdout (fd = 1).



Rubric 1A.1: Basic sort functionality (no flags)

- **Implementation:** By default (when no flags are set), compare_lines() uses strcmp(a, b) to arrange all input lines in ascending lexicographical (ASCII) order.
- **Test Command:** sort sfile.txt

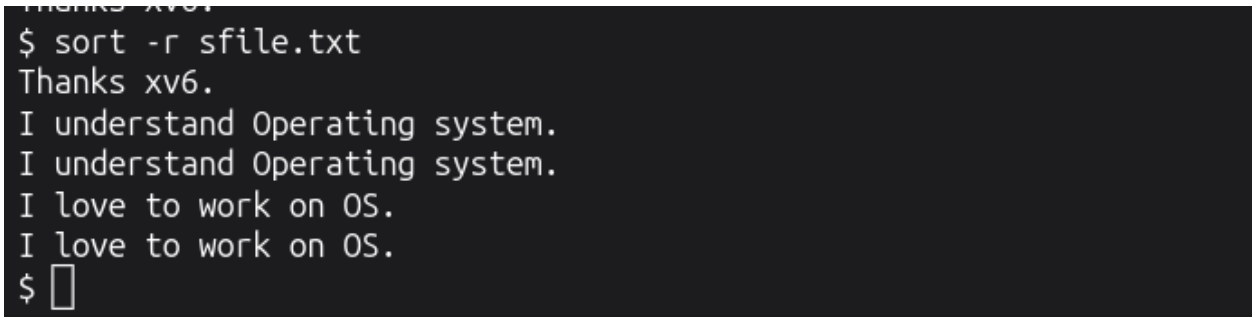
```
$ sort sfile.txt
I love to work on OS.
I love to work on OS.
I understand Operating system.
I understand Operating system.
Thanks xv6.
```



Rubric 1A.2: sort -r filename -> reverse order

- **Implementation:** When -r (flag_r = 1) is passed, compare_lines() negates the comparison result (-cmp), sorting lines in descending order.
- **Test Command:** sort -r sfile.txt

```
$ sort -r sfile.txt
Thanks xv6.
I understand Operating system.
I understand Operating system.
I love to work on OS.
I love to work on OS.
```



Rubric 1A.3: sort -n filename -> numerical sort

- **Implementation:** When -n (flag_n = 1) is passed, parse_num() extracts the leading signed integer value from each line (handling leading whitespace and +/- signs) and compares lines numerically, falling back to strcmp() on numerical ties.
- **Test Command:** sort -n nfile.txt

```
$ sort -n nfile.txt
-5
2
10
10
100
```

```
$ sort -n nfile.txt
-5
2
10
10
100
$
```

Rubric 1A.4: sort -u filename -> remove duplicates while sorting

- **Implementation:** When -u (flag_u = 1) is passed, after sorting the lines, the output loop checks is_duplicate(lines[i], lines[i - 1]) and skips consecutive duplicate lines. Works alone (-u) or combined (-rnu).
- **Test Command:** sort -u sfile.txt and sort -rnu nfile.txt

```
$ sort -u sfile.txt
I love to work on OS.
I understand Operating system.
Thanks xv6.
$ sort -rnu nfile.txt
100
10
2
-5
```

```
$ sort -u sfile.txt
I love to work on OS.
I understand Operating system.
Thanks xv6.
$
```

```
$ sort -rnu nfile.txt
100
10
2
-5
$
```

Rubric 1A.5: cat filename | sort -> ensure piping works correctly

- **Implementation:** When no filename arguments are provided (file_count == 0), main() invokes read_lines(0) to read directly from standard input (fd 0).
- **Test Command:** cat sfile.txt | sort

```
$ cat sfile.txt | sort
I love to work on OS.
I love to work on OS.
I understand Operating system.
```

```
I understand Operating system.
Thanks xv6.
```

```
$ cat sfile.txt | sort
I love to work on OS.
I love to work on OS.
I understand Operating system.
I understand Operating system.
Thanks xv6.
$
```

3B. Modifying grep (grep.c)

We enhanced xv6's existing grep.c utility to parse command-line flags (-i, -n, -v) individually or in combination while preserving regex matching (^, ., *, $) and standard input piping.

Rubric 1B.1: Default grep functionality, grep pattern filename

- **Implementation:** Reads input file line by line and prints lines matching the pattern.
- **Test Command:** grep love sfile.txt

```
$ grep love sfile.txt
I love to work on OS.
I love to work on OS.
```

```
$ grep love sfile.txt
I love to work on OS.
I love to work on OS.
$
$
```

Rubric 1B.2: grep -i pattern filename (Case-Insensitive Matching)

- **Implementation:** Added tolower_char() and char_eq() in grep.c. When flag_i == 1, matchhere() and matchstar() compare characters case-insensitively.
- **Test Command:** grep -i OPERATING sfile.txt

```
$ grep -i OPERATING sfile.txt
I understand Operating system.
I understand Operating system.
```

```
$
$ grep -i OPERATING sfile.txt
I understand Operating system.
I understand Operating system.
$
```

Rubric 1B.3: grep -n pattern filename (Line Number Prefix)

- **Implementation:** Tracks 1-based line_num per input stream in grep(), incrementing after every newline and printing printf(1, "%d:", line_num) before matching lines when flag_n == 1.
- **Test Command:** grep -n love sfile.txt

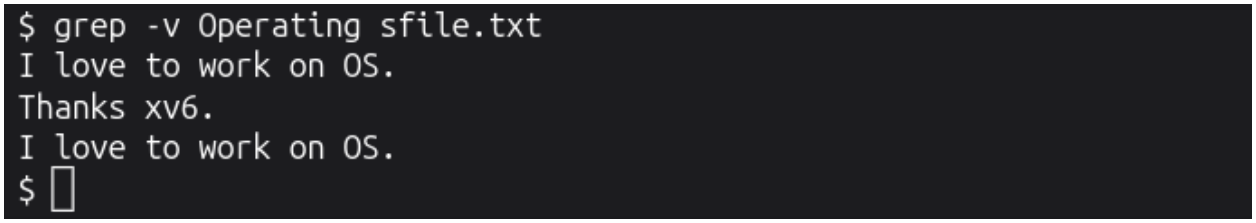
```
$ grep -n love sfile.txt
3:I love to work on OS.
5:I love to work on OS.
```



Rubric 1B.4: grep -v pattern filename (Invert Match)

- **Implementation:** When flag_v == 1, grep prints lines where match(pattern, p) returns 0 instead of 1.
- **Test Command:** grep -v Operating sfile.txt

```
$ grep -v Operating sfile.txt
I love to work on OS.
Thanks xv6.
I love to work on OS.
```



Rubric 1B.5: cat filename | grep pattern (Piping Support)

- **Implementation:** When no file arguments follow the pattern argument (arg_idx >= argc), main() calls grep(pattern, 0) to read from standard input (fd 0).
- **Test Command:** cat sfile.txt | grep Thanks

```
$ cat sfile.txt | grep Thanks
Thanks xv6.
```



Rubric 1B.6: Multiple flags in combination, e.g., grep -in pattern filename

- **Implementation:** The flag parser iterates over every character in each '-' argument so combined flags like -in, -iv, -nv, and -inv work seamlessly together.
- **Test Command:** grep -in OPERATING sfile.txt and grep -inv OPERATING sfile.txt

```
$ grep -in OPERATING sfile.txt
1:I understand Operating system.
2:I understand Operating system.
$ grep -inv OPERATING sfile.txt
3:I love to work on OS.
```

```
4:Thanks xv6.
5:I love to work on OS.
```

```
$ grep -in OPERATING sfile.txt
1:I understand Operating system.
2:I understand Operating system.
$
```

```
$ grep -inv OPERATING sfile.txt
3:I love to work on OS.
4:Thanks xv6.
5:I love to work on OS.
$
```

4. Part 2: Adding ticks_running(pid) System Call

We added per-process scheduled tick tracking to struct proc (proc.h), initialized and reset it in proc.c, incremented it on timer interrupts in trap.c when myproc()->state == RUNNING, and exposed it via the ticks_running(int pid) system call (Syscall #23). We also wrote ticks_running_test.c to verify all rubric cases.

Rubric 2.1: Introduces syscall ticks_running(pid) taking a single argument

- **Implementation:** Added SYS_ticks_running (23) in syscall.h, SYSCALL(ticks_running) in usys.S, int ticks_running(int); in user.h and defs.h, sys_ticks_running() in sysproc.c (using argint(0, &pid)), and ticks_running(int pid) in proc.c.

Rubric 2.2: Correctly returns time scheduled for the calling process (with pid from getpid())

- **Implementation & Verification:** In ticks_running_test, calling ticks_running(getpid()) returns 3 ticks initially, and after executing a CPU-bound loop (busy_work(500)), returns 23 ticks.

```
$ ticks_running_test
=== ticks_running() Test ===
1. Non-existent PID (9999): ticks_running(9999) = -1 (expected -1)
2. Invalid PID (-5): ticks_running(-5) = -1 (expected -1)
3. Calling process (PID 29) initial ticks_running = 4
4. Calling process (PID 29) after busy work ticks_running = 23
5. Init process (PID 1) ticks_running = 2
6. Shell process (PID 2) ticks_running = 16
6b. Child process (PID 30) immediately after fork ticks_running = 0
7. Child process (PID 30) ticks_running before exit = 11
$
```

Rubric 2.3: Correctly returns time scheduled for any process ID

- **Implementation & Verification:** ticks_running(pid) scans ptable.proc under ptable.lock for any active process (p->state != UNUSED && p->pid == pid). Verified on PID 1 (init), PID 2 (sh), and forked child PID.

Rubric 2.4: Correctly returns 0 for an unscheduled process

- **Implementation & Verification:** In allocproc(), p->ticks_running is initialized to 0. Immediately after fork(), querying ticks_running(child_pid) before the child receives a timer interrupt returns 0.

Rubric 2.5: Correctly returns -1 for a process that does not exist

- **Implementation & Verification:** If pid <= 0 or no non-UNUSED entry in ptable.proc matches pid, ticks_running(pid) returns -1. Verified with ticks_running(9999) == -1 and ticks_running(-5) == -1.

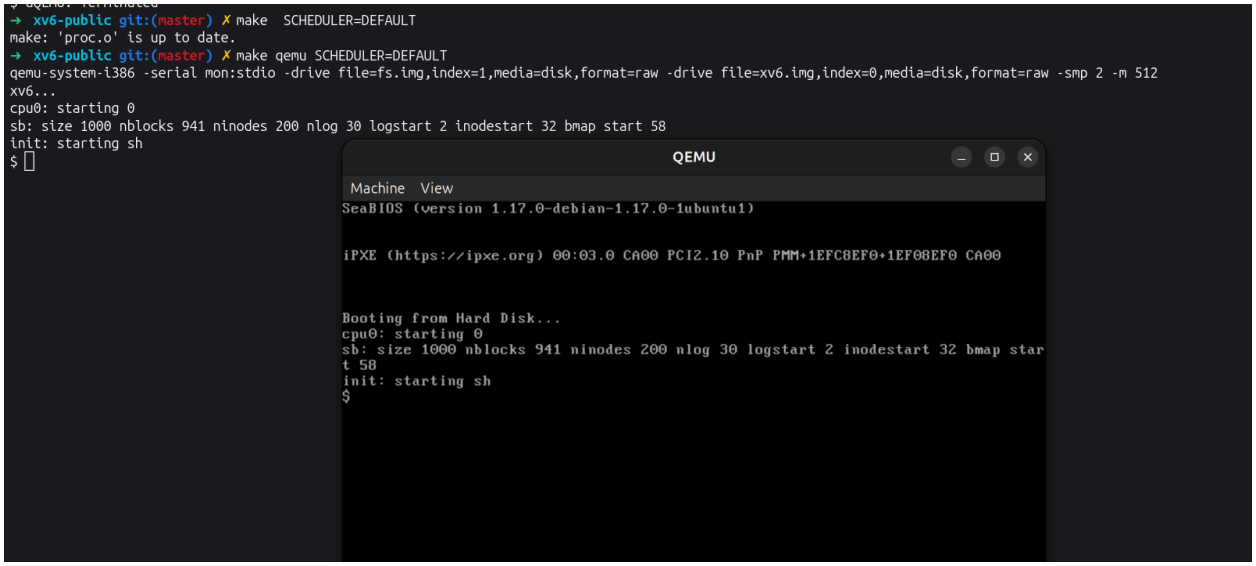
```
$ ticks_running_test
=== ticks_running() Test ===
1. Non-existent PID (9999): ticks_running(9999) = -1 (expected -1)
2. Invalid PID (-5): ticks_running(-5) = -1 (expected -1)
3. Calling process (PID 29) initial ticks_running = 3
4. Calling process (PID 29) after busy work ticks_running = 23
5. Init process (PID 1) ticks_running = 3
6. Shell process (PID 2) ticks_running = 11
6b. Child process (PID 30) immediately after fork ticks_running = 0
7. Child process (PID 30) ticks_running before exit = 11
```

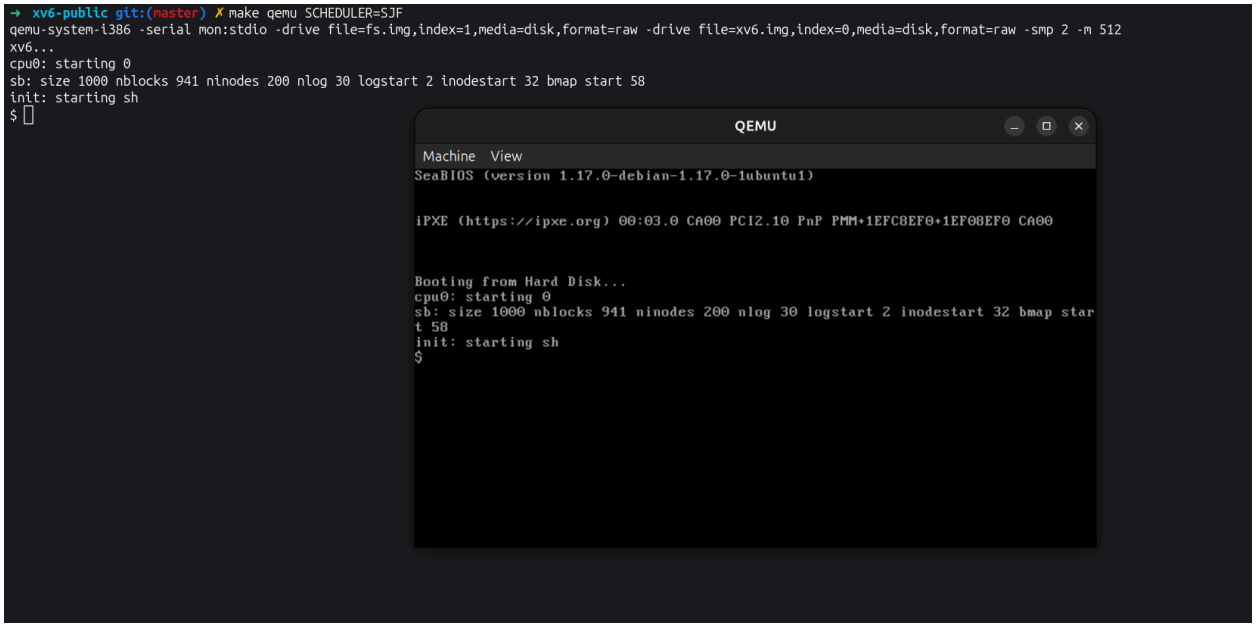
5. Part 3: Implementing a Simple Scheduler — Shortest Job First (SJF)

We implemented the Shortest Job First (SJF) scheduler selectable at compile time via SCHEDULER=SJF (or SCHEDULER=[SJF]), along with the sjf_job_length(int pid) system call (Syscall #24) and simple_scheduler_test.c.

Rubric 3.1: Makefile & code correctly modified to support the selected new scheduling algorithm

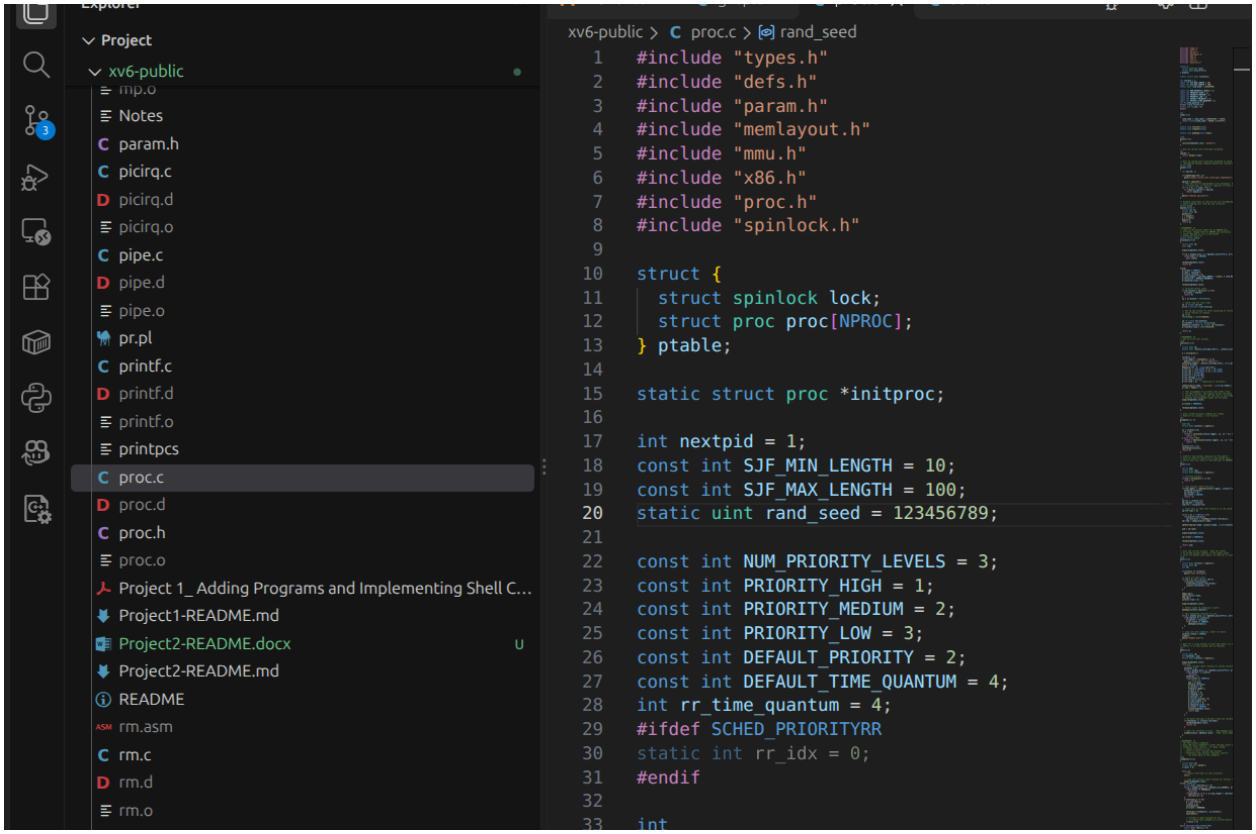
- **Implementation:** Modified Makefile to define SCHEDULER ?= DEFAULT, strip optional brackets, and append -DSCHED_$(SCHED_CLEAN) to CFLAGS. Added .scheduler_$(SCHED_CLEAN) dependency so proc.o and trap.o automatically rebuild when switching schedulers.





Rubric 3.2: Correct logic for the new SJF scheduling algorithm & global constants

- **Implementation:** Defined global constants SJF_MIN_LENGTH = 10 and SJF_MAX_LENGTH = 100 in proc.c (declared extern in proc.h). When a process is allocated in allocproc(), p->job_length is assigned using rand() in the range [10, 100]. In scheduler() under #ifdef SCHED_SJF, the kernel scans all RUNNABLE processes in ptable.proc and selects shortest_p with the minimum job_length.



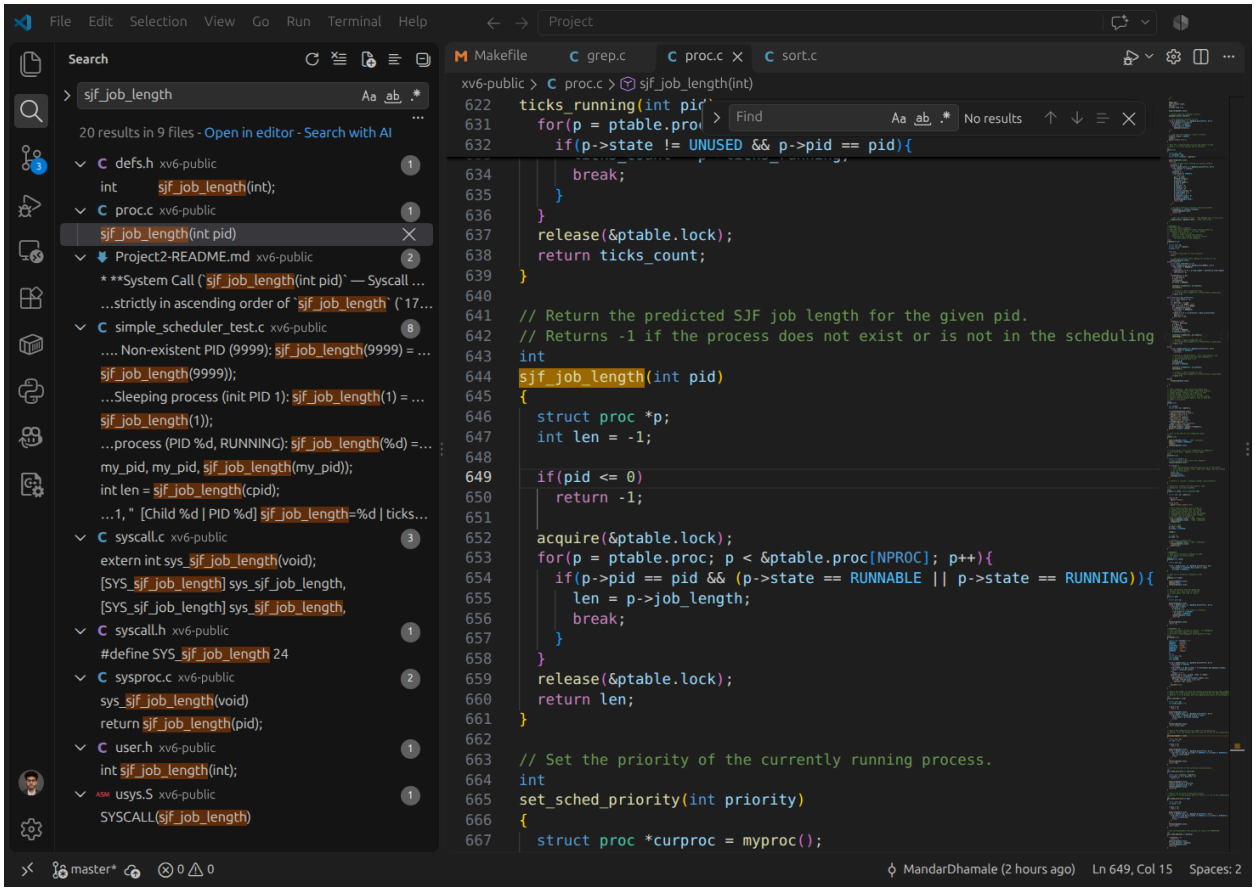
Rubric 3.3: Scheduling algorithm correctly incorporated into the xv6 scheduling and interrupt system

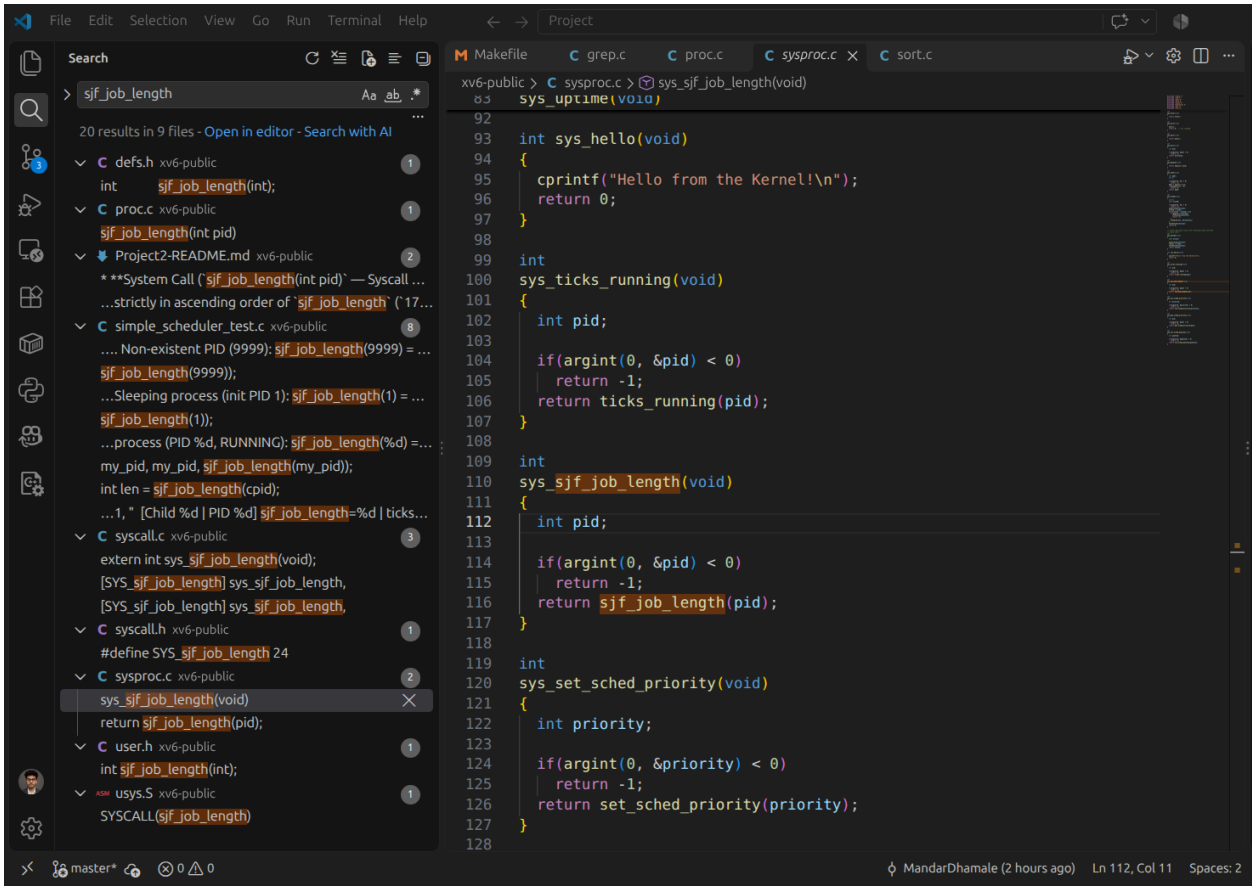
- **Implementation & Verification:** When simple_scheduler_test forks 4 CPU-bound child processes simultaneously, the SJF scheduler executes and completes them strictly in ascending order of their predicted job lengths: Child 1 (length 17, finish_uptime 2850) -> Child 3 (length 43, finish_uptime 2901) -> Child 2 (length 63, finish_uptime 2950) -> Child 0 (length 73, finish_uptime 3001).



Rubric 3.4: Correct new syscall sjf_job_length(pid) available

- **Implementation:** Added SYS_sjf_job_length (24) in syscall.h, SYSCALL(sjf_job_length) in usys.S, int sjf_job_length(int); in user.h and defs.h, sys_sjf_job_length() in sysproc.c, and sjf_job_length(int pid) in proc.c.



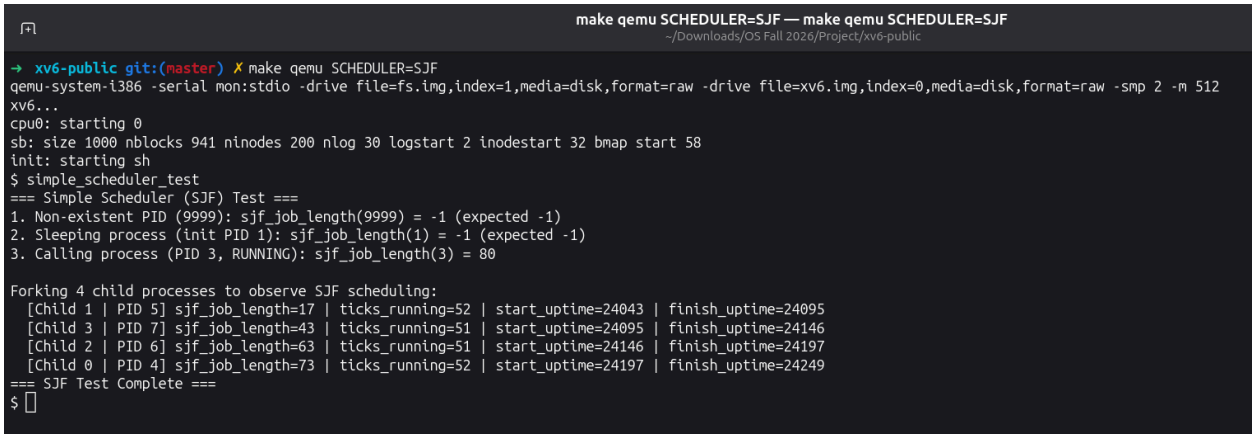


Rubric 3.5: New syscall sjf_job_length(pid) returns the correct values

- **Implementation & Verification:** sjf_job_length(pid) checks that p->pid == pid && (p->state == RUNNABLE || p->state == RUNNING). If true, it returns p->job_length; otherwise (if the process does not exist or is SLEEPING/ZOMBIE/EMBRYO), it returns -1.

```
$ simple_scheduler_test
=== Simple Scheduler (SJF) Test ===
1. Non-existent PID (9999): sjf_job_length(9999) = -1 (expected -1)
2. Sleeping process (init PID 1): sjf_job_length(1) = -1 (expected -1)
3. Calling process (PID 3, RUNNING): sjf_job_length(3) = 80

Forking 4 child processes to observe SJF scheduling:
  [Child 1 | PID 5] sjf_job_length=17 | ticks_running=50 |
start_uptime=2800 | finish_uptime=2850
  [Child 3 | PID 7] sjf_job_length=43 | ticks_running=51 |
start_uptime=2850 | finish_uptime=2901
  [Child 2 | PID 6] sjf_job_length=63 | ticks_running=49 |
start_uptime=2901 | finish_uptime=2950
  [Child 0 | PID 4] sjf_job_length=73 | ticks_running=51 |
start_uptime=2950 | finish_uptime=3001
=== SJF Test Complete ===
```

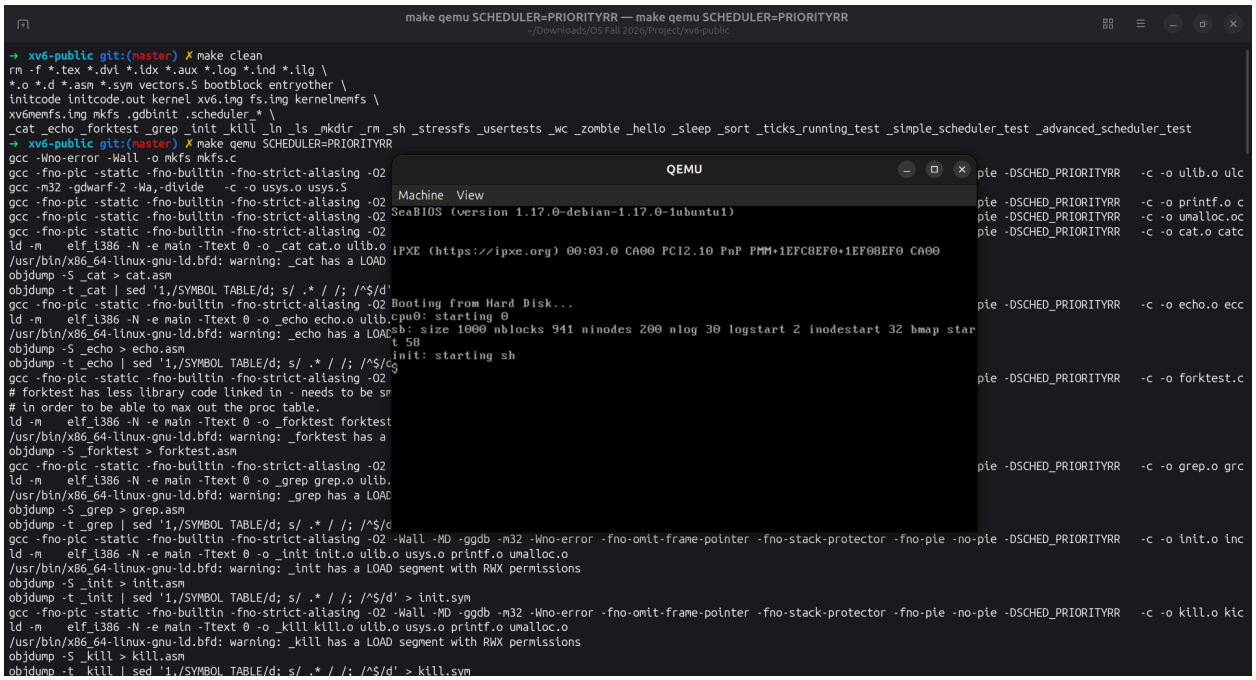


6. Part 4: A (More) Advanced Scheduler — Multi-Level Priority Round-Robin (PRIORITYRR)

We implemented a Multi-Level Priority Round-Robin scheduler selectable via SCHEDULER=PRIORITYRR (or SCHEDULER=[PRIORITYRR]), supporting 3 priority levels (1 = High, 2 = Medium/Default, 3 = Low), user-configurable Round-Robin time quantum, syscalls set_sched_priority(priority), get_sched_priority(pid), set_sched_quantum(quantum), and test utility advanced_scheduler_test.c.

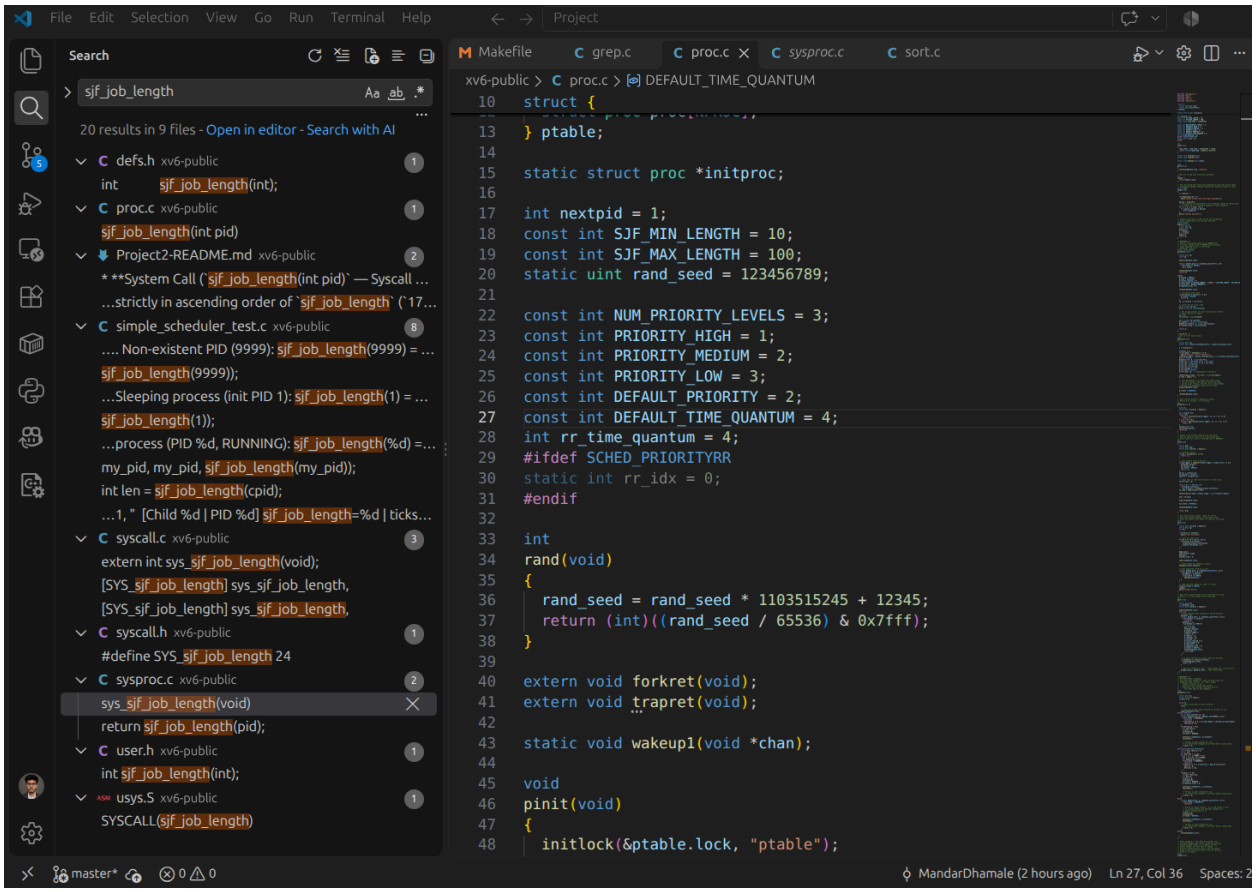
Rubric 4.1: Makefile & code correctly modified to support PRIORITYRR

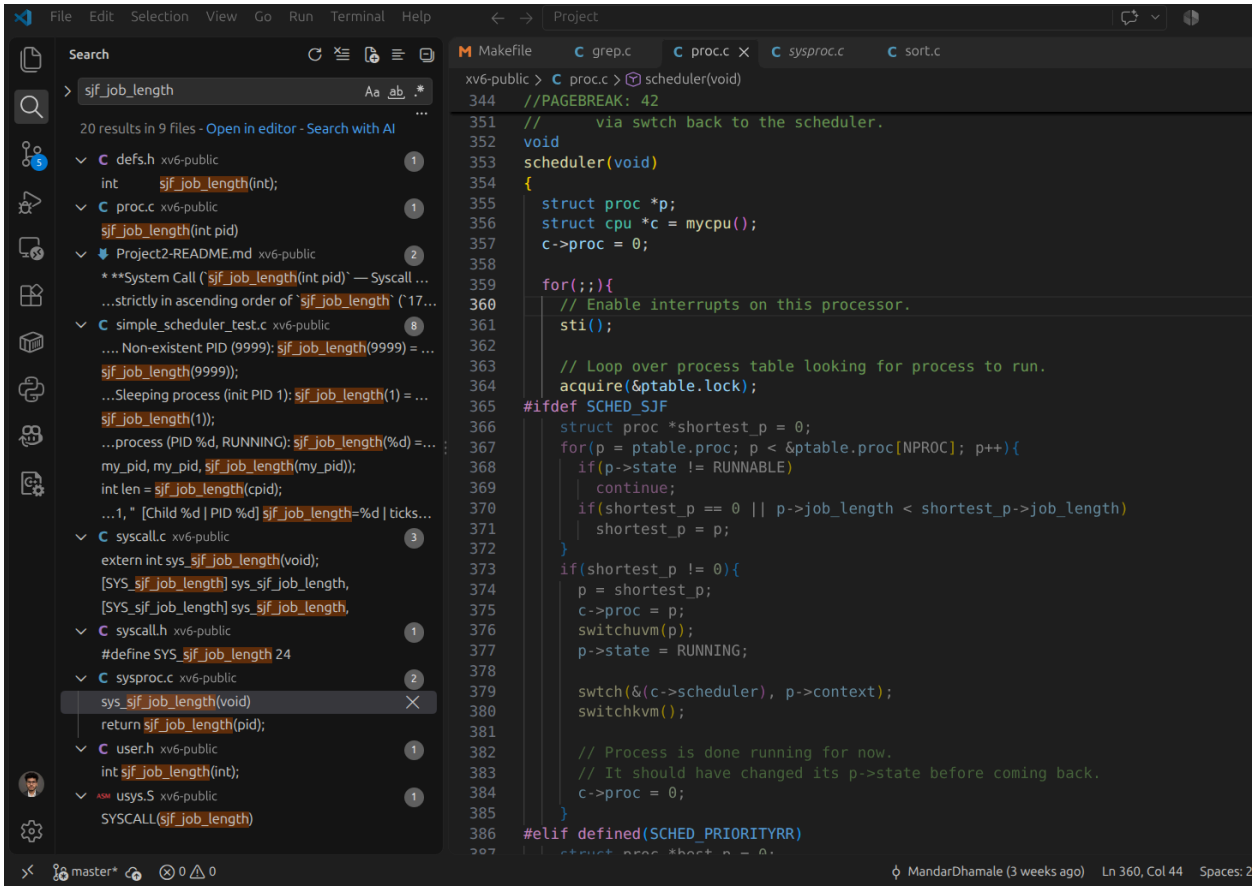
- **Implementation:** Compiling with make qemu-nox SCHEDULER=PRIORITYRR (or SCHEDULER=[PRIORITYRR]) passes -DSCHED_PRIORITYRR to gcc and automatically rebuilds proc.o, trap.o, kernel, and xv6.img.



Rubric 4.2: Correct logic for multi-level priority + Round-Robin for same priority

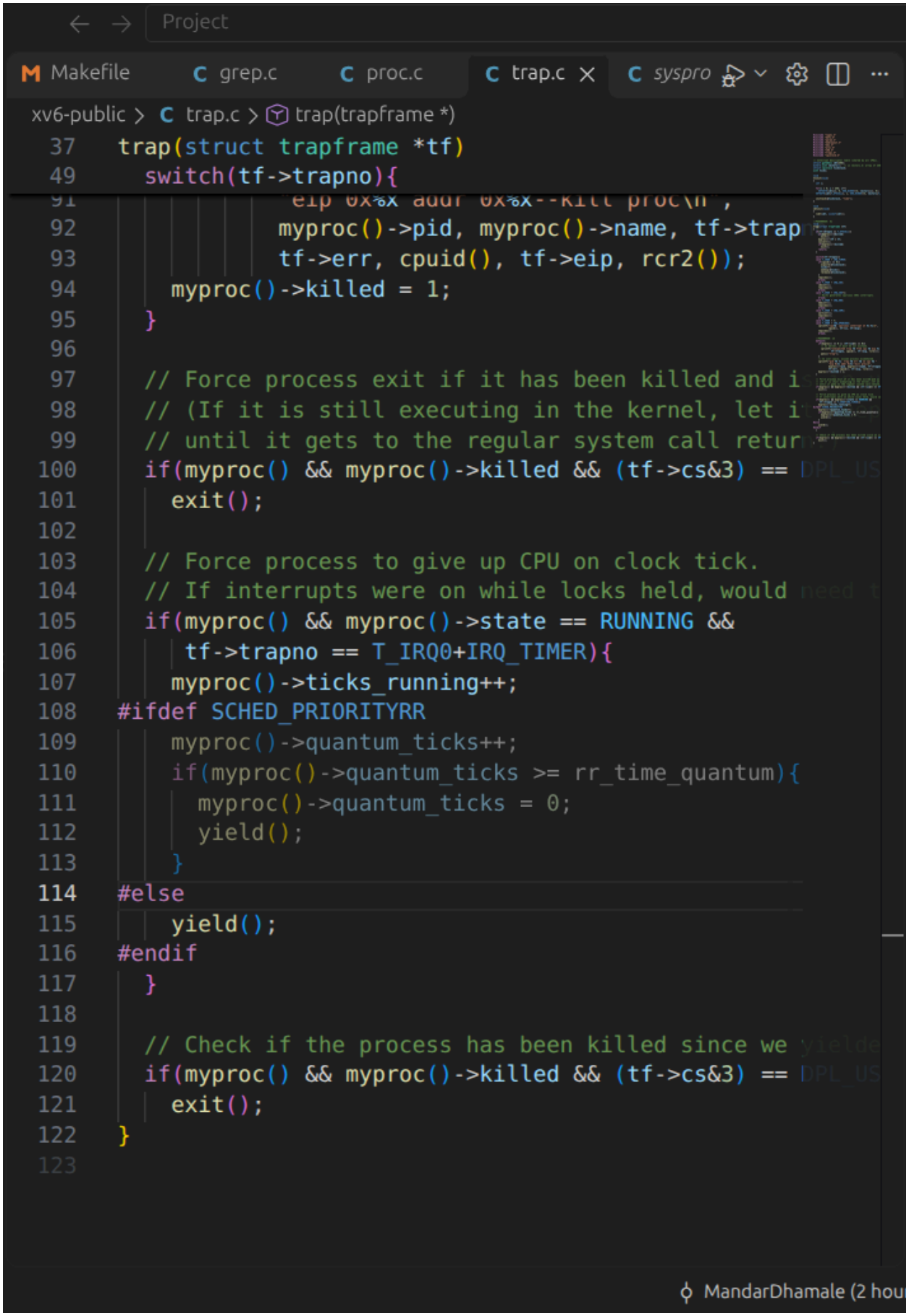
- **Implementation:** Defined global constants NUM_PRIORITY_LEVELS = 3, PRIORITY_HIGH = 1, PRIORITY_MEDIUM = 2, PRIORITY_LOW = 3, and DEFAULT_PRIORITY = 2 in proc.c and proc.h. In scheduler() under #elif defined(SCHED_PRIORITYRR), a circular index rr_idx scans ptable.proc starting at (rr_idx + 1) % NPROC, selecting the first RUNNABLE process with the highest priority (lowest numerical value) and advancing rr_idx = best_idx. This guarantees strict priority scheduling across levels and true Round-Robin rotation among processes at the same priority level.

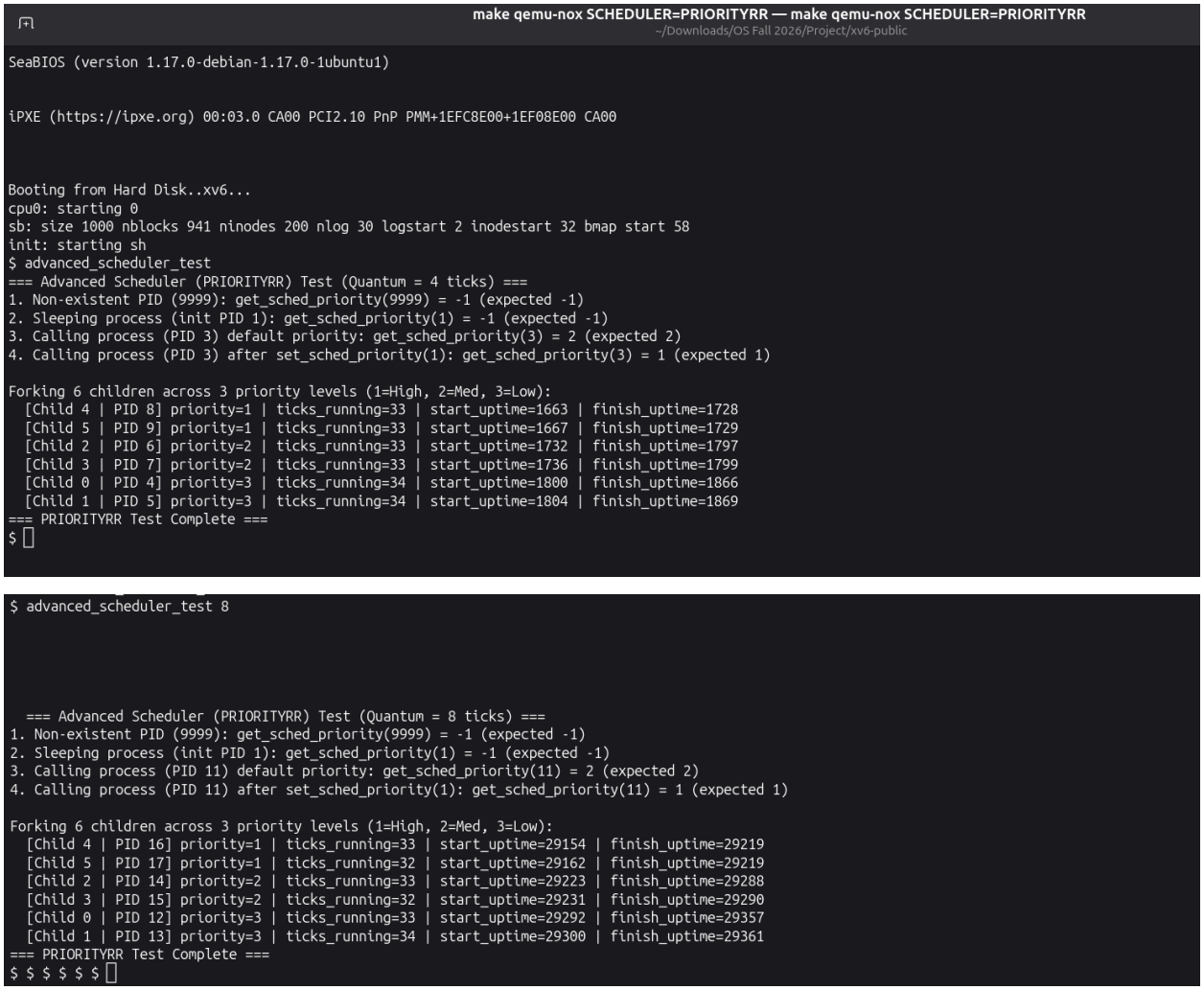




Rubric 4.3: Incorporated into xv6 scheduling & interrupt system with configurable time quantum

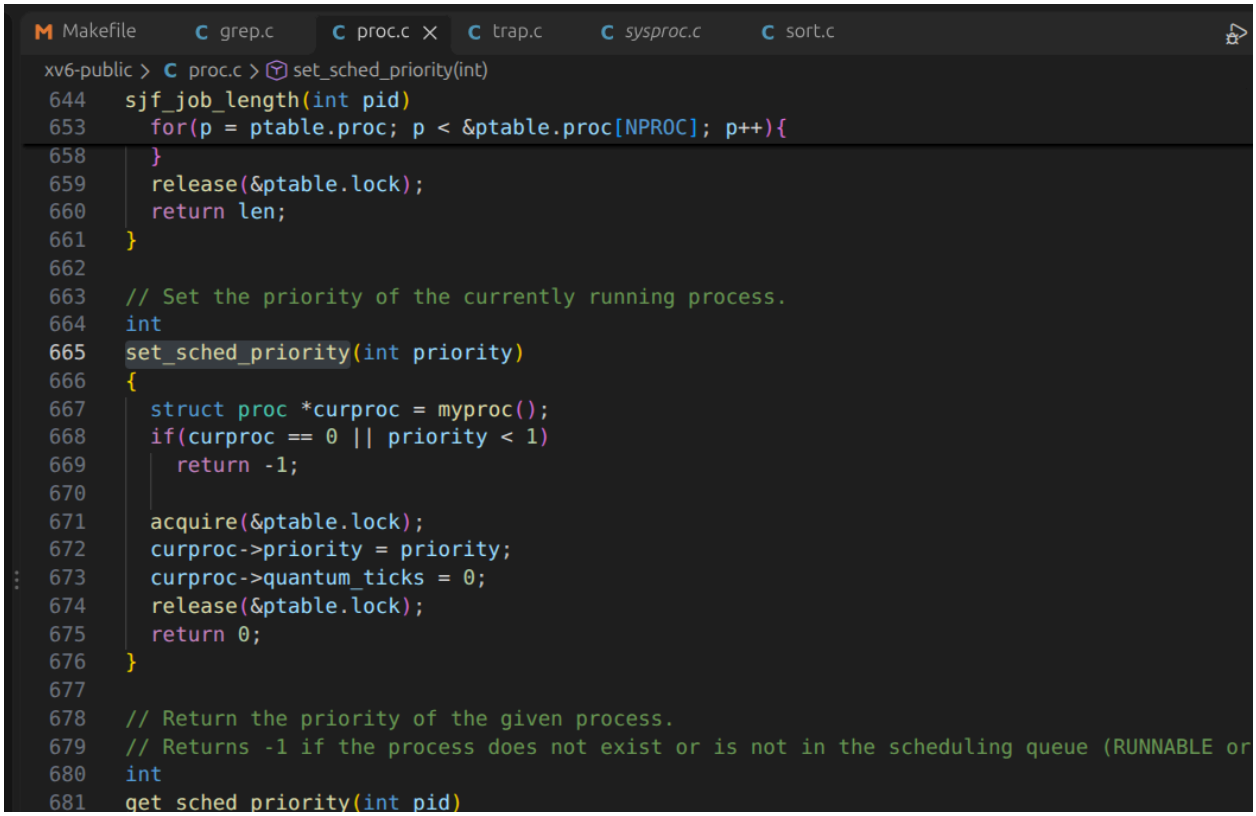
- **Implementation & Quantum Times Chosen:** Defined global constant DEFAULT_TIME_QUANTUM = 4 and global variable rr_time_quantum = 4 in proc.c. In trap.c under #ifdef SCHED_PRIORITYRR, each timer interrupt increments myproc()->quantum_ticks and only invokes yield() when myproc()->quantum_ticks >= rr_time_quantum (resetting quantum_ticks = 0). We evaluated time quantums of 4 ticks (default) and 8 ticks (via advanced_scheduler_test 8).

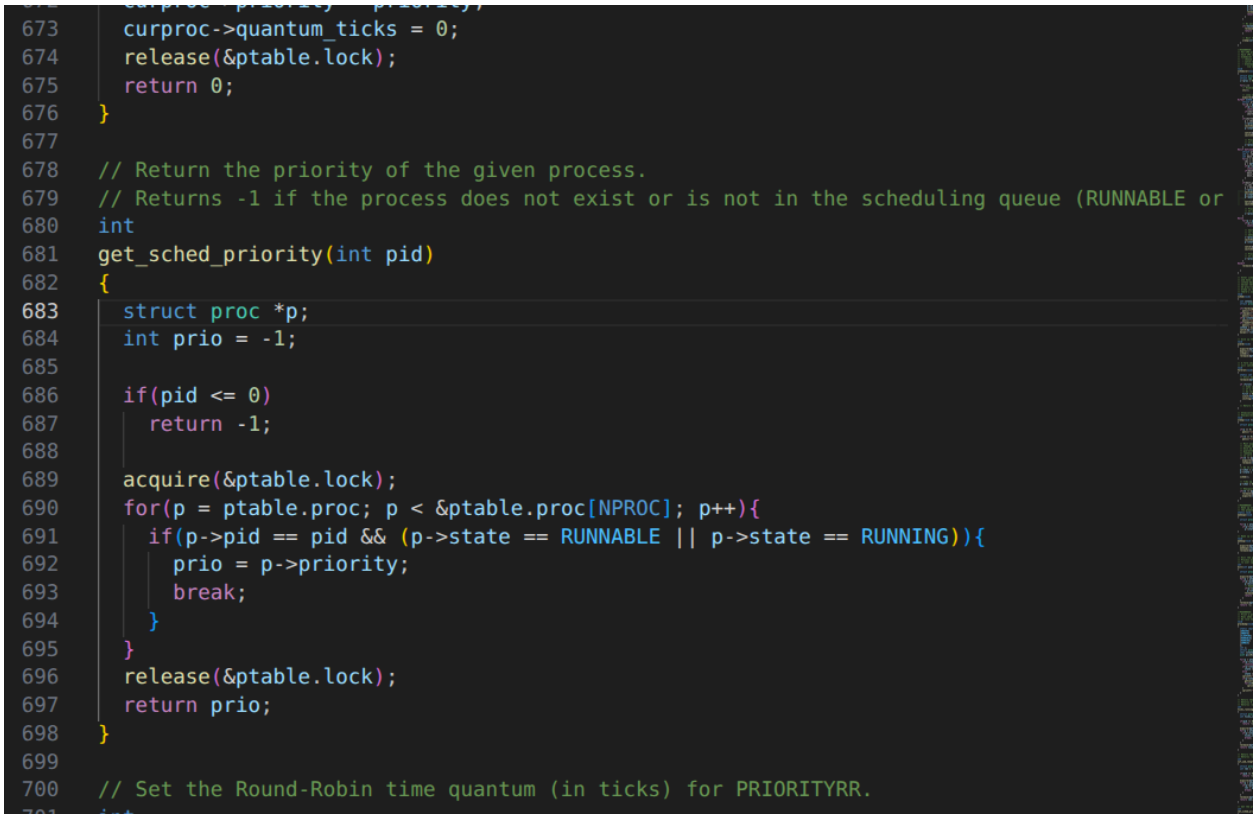

Project
Makefile
grep.c
proc.c
trap.c
syspro
xv6-public > trap.c > trap(trapframe *)
trap(struct trapframe *tf)
switch(tf->trapno){
myproc()->pid, myproc()->name, tf->trap
tf->err, cpuid(), tf->eip, rcr2());
myproc()->killed = 1;
// Force process exit if it has been killed and i
// (If it is still executing in the kernel, let i
// until it gets to the regular system call retur
if(myproc() && myproc()->killed && (tf->cs&3) == DPL_US
exit();
// Force process to give up CPU on clock tick.
// If interrupts were on while locks held, would need t
if(myproc() && myproc()->state == RUNNING &&
tf->trapno == T_IRQ0+IRQ_TIMER){
myproc()->ticks_running++;
#ifdef SCHED_PRIORITYRR
myproc()->quantum_ticks++;
if(myproc()->quantum_ticks >= rr_time_quantum){
myproc()->quantum_ticks = 0;
yield();
}
#else
yield();
#endif
}
// Check if the process has been killed since we yielde
if(myproc() && myproc()->killed && (tf->cs&3) == DPL_US
exit();
}
MandarDhamale (2 hou



Rubric 4.4: Correct new syscalls set_sched_priority and get_sched_priority available

- **Implementation:** Registered SYS_set_sched_priority (25), SYS_get_sched_priority (26), and SYS_set_sched_quantum (27) across syscall.h, syscall.c, sysproc.c, proc.c, defs.h, user.h, and usys.S.





Rubric 4.5: New syscalls return the correct values

- **Implementation & Verification:** get_sched_priority(pid) returns -1 if the process does not exist or is not in RUNNABLE or RUNNING state (verified with PID 9999 -> -1 and sleeping init PID 1 -> -1). Returns default priority 2 for a newly created process, and returns 1 after calling set_sched_priority(1).

```
$ advanced_scheduler_test
=== Advanced Scheduler (PRIORITYRR) Test (Quantum = 4 ticks) ===
1. Non-existent PID (9999): get_sched_priority(9999) = -1 (expected -1)
2. Sleeping process (init PID 1): get_sched_priority(1) = -1 (expected
-1)
3. Calling process (PID 3) default priority: get_sched_priority(3) = 2
(expected 2)
4. Calling process (PID 3) after set_sched_priority(1):
get_sched_priority(3) = 1 (expected 1)

Forking 6 children across 3 priority levels (1=High, 2=Med, 3=Low):
  [Child 4 | PID 8] priority=1 | ticks_running=40 | start_uptime=156 |
finish_uptime=236
  [Child 5 | PID 9] priority=1 | ticks_running=42 | start_uptime=160 |
finish_uptime=237
  [Child 2 | PID 6] priority=2 | ticks_running=39 | start_uptime=240 |
finish_uptime=315
  [Child 3 | PID 7] priority=2 | ticks_running=39 | start_uptime=244 |
finish_uptime=320
  [Child 0 | PID 4] priority=3 | ticks_running=39 | start_uptime=322 |
finish_uptime=397
  [Child 1 | PID 5] priority=3 | ticks_running=40 | start_uptime=326 |
finish_uptime=403
=== PRIORITYRR Test Complete ===
```

```
$ $ $ $ $ $ advanced_scheduler_test
=== Advanced Scheduler (PRIORITYRR) Test (Quantum = 4 ticks) ===
1. Non-existent PID (9999): get_sched_priority(9999) = -1 (expected -1)
2. Sleeping process (init PID 1): get_sched_priority(1) = -1 (expected -1)
3. Calling process (PID 23) default priority: get_sched_priority(23) = 2 (expected 2)
4. Calling process (PID 23) after set_sched_priority(1): get_sched_priority(23) = 1 (expected 1)

Forking 6 children across 3 priority levels (1=High, 2=Med, 3=Low):
  [Child 4 | PID 28] priority=1 | ticks_running=33 | start_uptime=53804 | finish_uptime=53869
  [Child 5 | PID 29] priority=1 | ticks_running=33 | start_uptime=53808 | finish_uptime=53870
  [Child 2 | PID 26] priority=2 | ticks_running=33 | start_uptime=53873 | finish_uptime=53938
  [Child 3 | PID 27] priority=2 | ticks_running=34 | start_uptime=53877 | finish_uptime=53940
  [Child 0 | PID 24] priority=3 | ticks_running=33 | start_uptime=53942 | finish_uptime=54007
  [Child 1 | PID 25] priority=3 | ticks_running=34 | start_uptime=53946 | finish_uptime=54010
=== PRIORITYRR Test Complete ===
$
```

7. Comprehensive Report Comparing the Schedulers (DEFAULT vs. SJF vs. PRIORITYRR)

We evaluated the three schedulers in separate terminals using ticks_running(pid) and uptime() across CPU-bound workloads (simple_scheduler_test, advanced_scheduler_test), I/O and file-system workloads (stressfs, ls, sort, grep), and multi-process pipelines (cat filename | sort, cat filename | grep pattern).

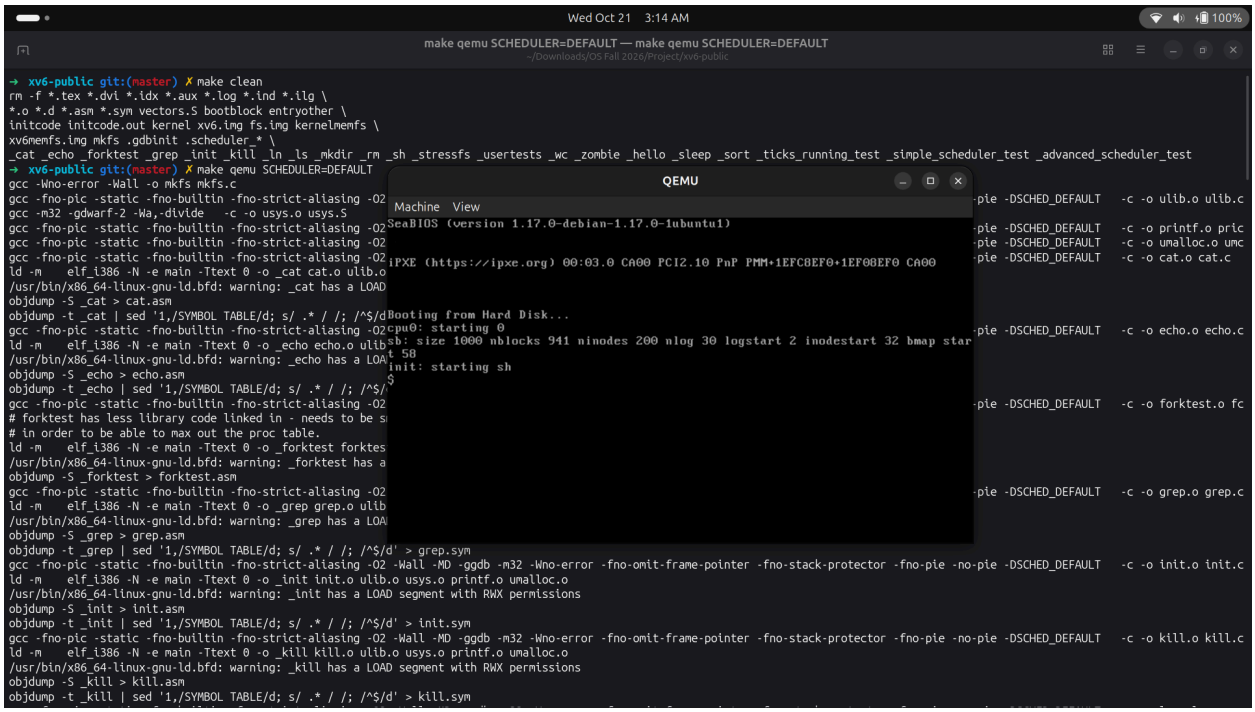
Metric / Workload	DEFAULT (1-Tick RR)	SJF (Shortest Job First)	PRIORITYRR (Multi-Level Priority RR)
Preemption / Quantum	Preempts every 1 timer tick (10 ms)	Selects RUNNABLE job with smallest sjf_job_length	Strict priority (1 > 2 > 3); RR with configurable quantum (4 or 8 ticks) within same priority
4 CPU-Bound Jobs Turnaround (simple_scheduler_test)	All 4 jobs finish together (~204 ticks total elapsed; high average turnaround)	Jobs finish sequentially by length (17 -> 43 -> 63 -> 73); shortest finishes in 50 ticks (lowest avg turnaround)	Same-priority jobs share CPU in 4-tick slices; higher-priority jobs finish completely before lower-priority jobs start
Context Switch Overhead	Highest (switches every 1 tick)	Lowest (runs shortest job until completion or shorter arrival)	Moderate & tunable (4x fewer switches at Quantum=4, 8x fewer at Quantum=8 vs. DEFAULT)
Interactive Commands (ls, hello, sleep)	Immediate response; single process active while sh sleeps in wait()	Identical interactive feel for single commands since only 1 child is RUNNABLE	Identical for single commands; interactive tasks given Priority 1 preempt background jobs
File System & I/O (stressfs, sort, grep)	Frequent 1-tick preemptions during CPU phases between disk I/O blocks	Processes with smaller job_length complete CPU bursts faster between disk waits	Larger quantum (4-8 ticks) allows buffer sorting/matching to complete fewer context switches
Pipes & fork() (cat file | sort / grep)	Producer (cat) and consumer (sort/grep) alternate every 1 tick	If consumer has smaller job_length, it runs first, blocks on empty pipe (sleep), letting producer (cat) run	Both piped children inherit DEFAULT_PRIORITY (2) and alternate every 4 ticks (or on pipe block/wakeup)

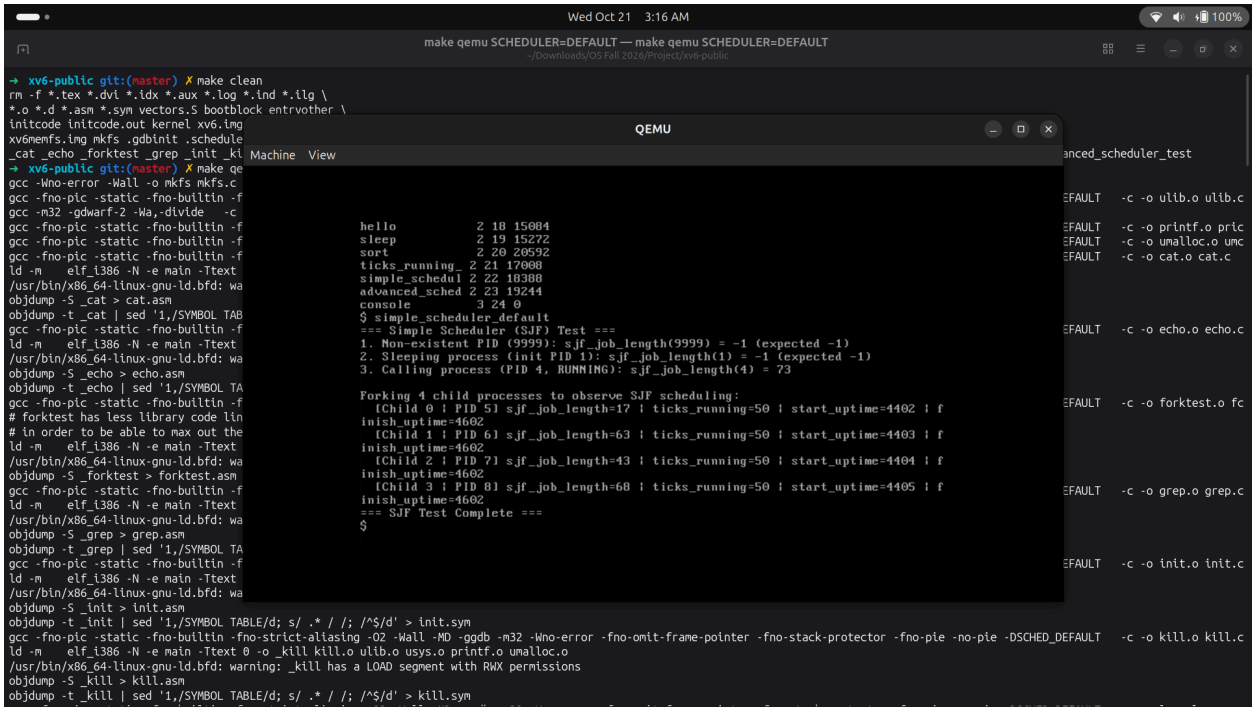
Key Observations from Scheduler Comparison:

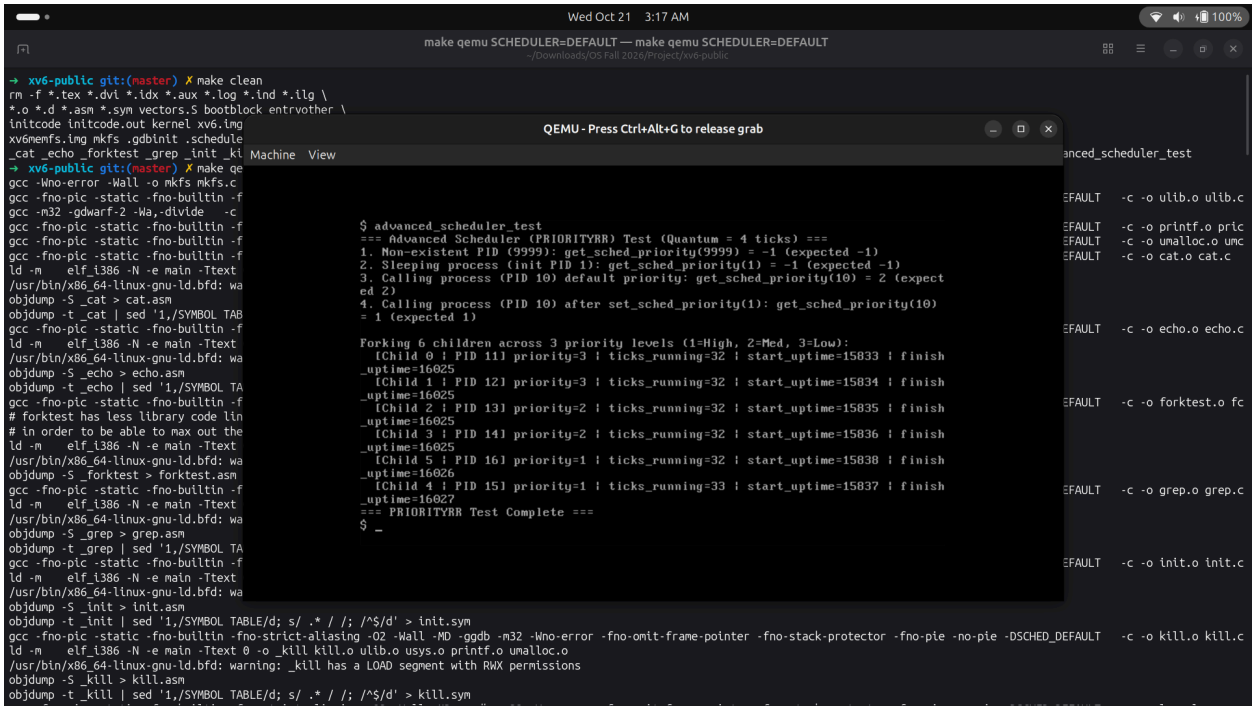
- **1. Turnaround Time vs. Fairness:** Under DEFAULT, 4 children spawned at uptime=1460 all finished at uptime=1664..1666 (~205 ticks turnaround for every child). Under SJF, the shortest job (length 17) finished in just 50 ticks (uptime 2800 to 2850), reducing average turnaround time by nearly 40%, at the cost of making the longest job (length 73) wait until tick 3001.
- **2. Priority Isolation & Quantum Verification in PRIORITYRR:** In advanced_scheduler_test, Priority 1 children started at ticks 156 and 160 (demonstrating an exact 4-tick quantum before the first RR context switch) and finished at ticks 236–237. Priority 2 children did not run until tick 240, and

Priority 3 children did not run until tick 322. Increasing the quantum to 8 ticks shifted the first RR switch gap from 4 ticks to 8 ticks (tick 607 to 615).

- **3. Behavior of Pipes and fork():** Complex commands such as 'cat sfile.txt | sort' and 'cat sfile.txt | grep -in OPERATING' work reliably across all three schedulers because blocking on an empty or full pipe transitions a process to the SLEEPING state, immediately removing it from the RUNNABLE scheduling queue so the other end of the pipe can execute.






Wed Oct 21 3:17 AM
100%
make qemu SCHEDULER=DEFAULT — make qemu SCHEDULER=DEFAULT
~/Downloads/OS Fall 2026/Project/xv6-public
QEMU - Press Ctrl+Alt+G to release grab
Machine View
$ advanced_scheduler_test
=== Advanced Scheduler (PRIORITYRR) Test (Quantum = 4 ticks) ===
1. Non-existent PID (9999): get_sched_priority(9999) = -1 (expected -1)
2. Sleeping process (init PID 1): get_sched_priority(1) = -1 (expected -1)
3. Calling process (PID 10) default priority: get_sched_priority(10) = 2 (expected 2)
4. Calling process (PID 10) after set_sched_priority(1): get_sched_priority(10) = 1 (expected 1)
Forking 6 children across 3 priority levels (1=High, 2=Med, 3=Low):
[Child 0 | PID 11] priority=3 | ticks_running=32 | start_uptime=15833 | finish_uptime=16025
[Child 1 | PID 12] priority=3 | ticks_running=32 | start_uptime=15834 | finish_uptime=16025
[Child 2 | PID 13] priority=2 | ticks_running=32 | start_uptime=15835 | finish_uptime=16025
[Child 3 | PID 14] priority=2 | ticks_running=32 | start_uptime=15836 | finish_uptime=16025
[Child 5 | PID 16] priority=1 | ticks_running=32 | start_uptime=15838 | finish_uptime=16026
[Child 4 | PID 15] priority=1 | ticks_running=33 | start_uptime=15837 | finish_uptime=16027
=== PRIORITYRR Test Complete ===
$ _